TASK 3

Iris Flower Classification

Objective

Develop a machine-learning model that classifies Iris flowers into their species using sepal and petal measurements.

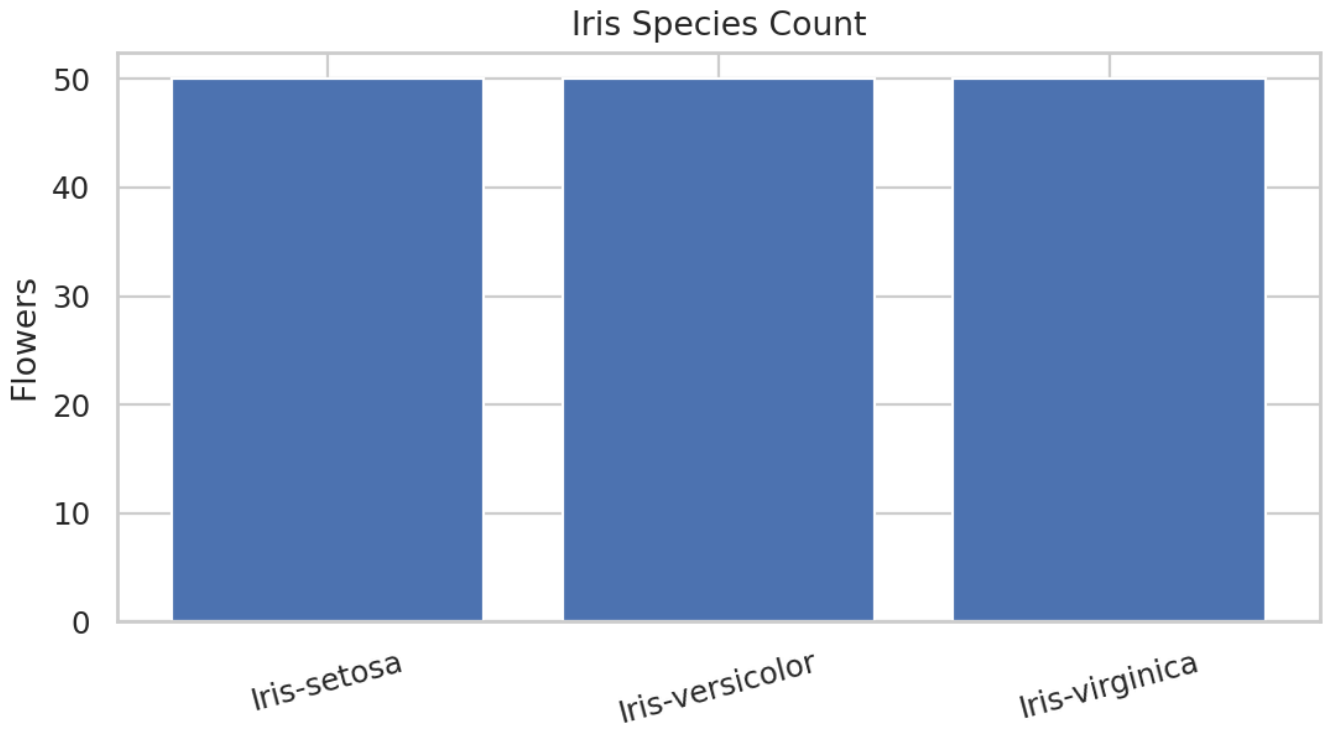
Dataset

The uploaded CodSoft brief defines Task 3 as Iris Flower Classification and specifies setosa, versicolor and virginica.

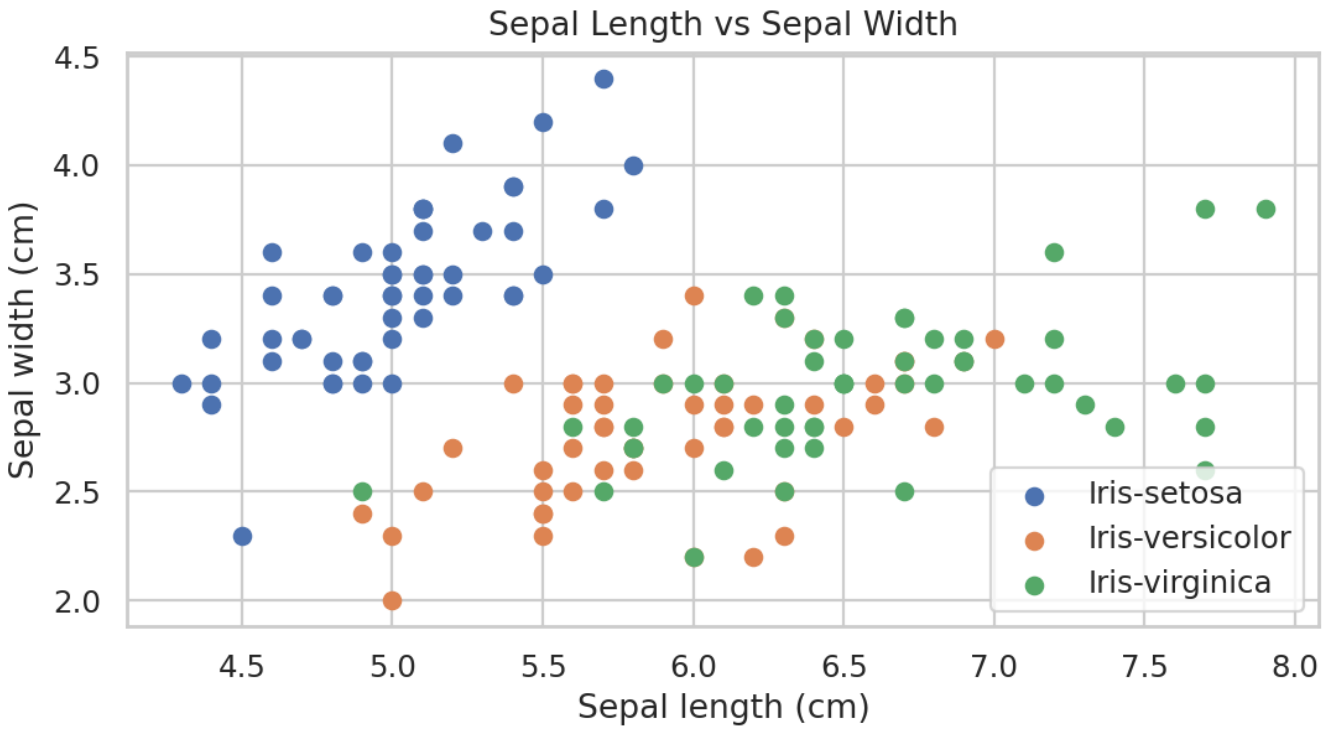
Tools / Libraries

Library	Purpose
Python	Programming
Pandas / NumPy	Data processing
Matplotlib / Seaborn	Data visualization
Scikit-learn	Machine learning

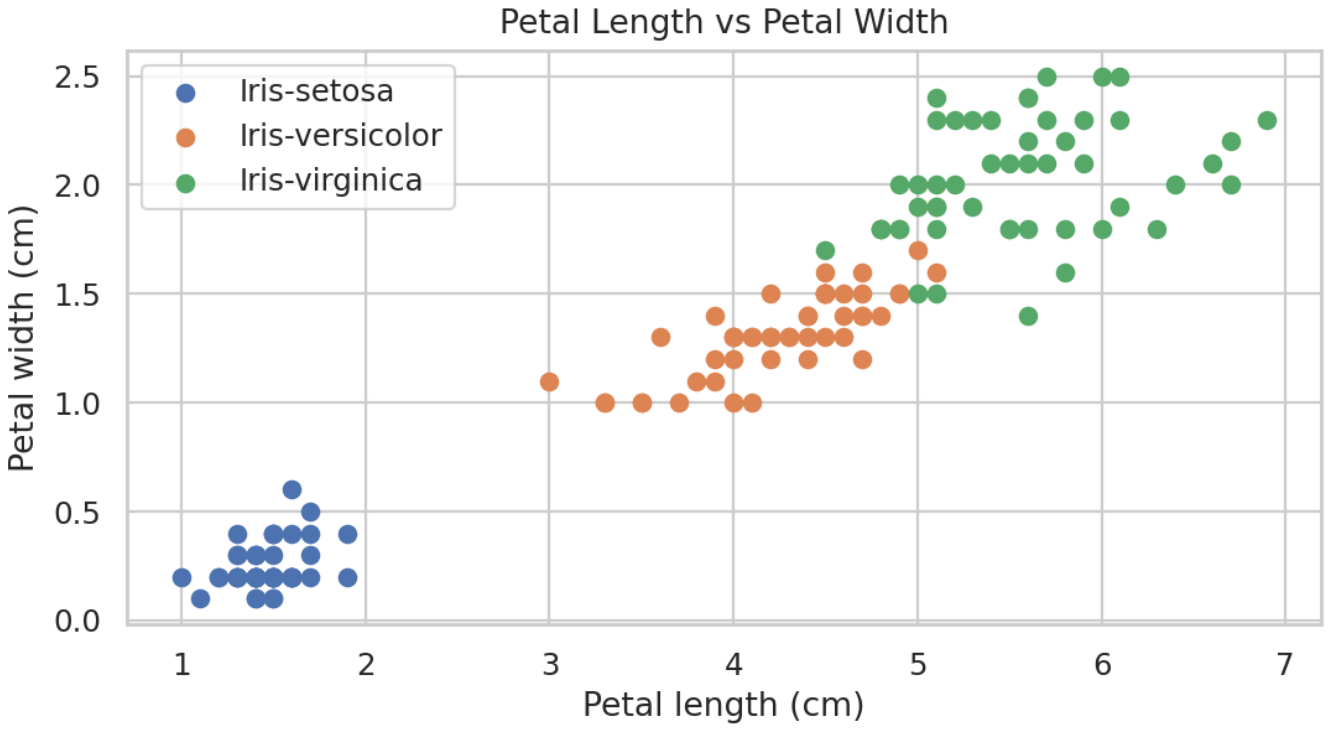
Visualization 1 – Species Count



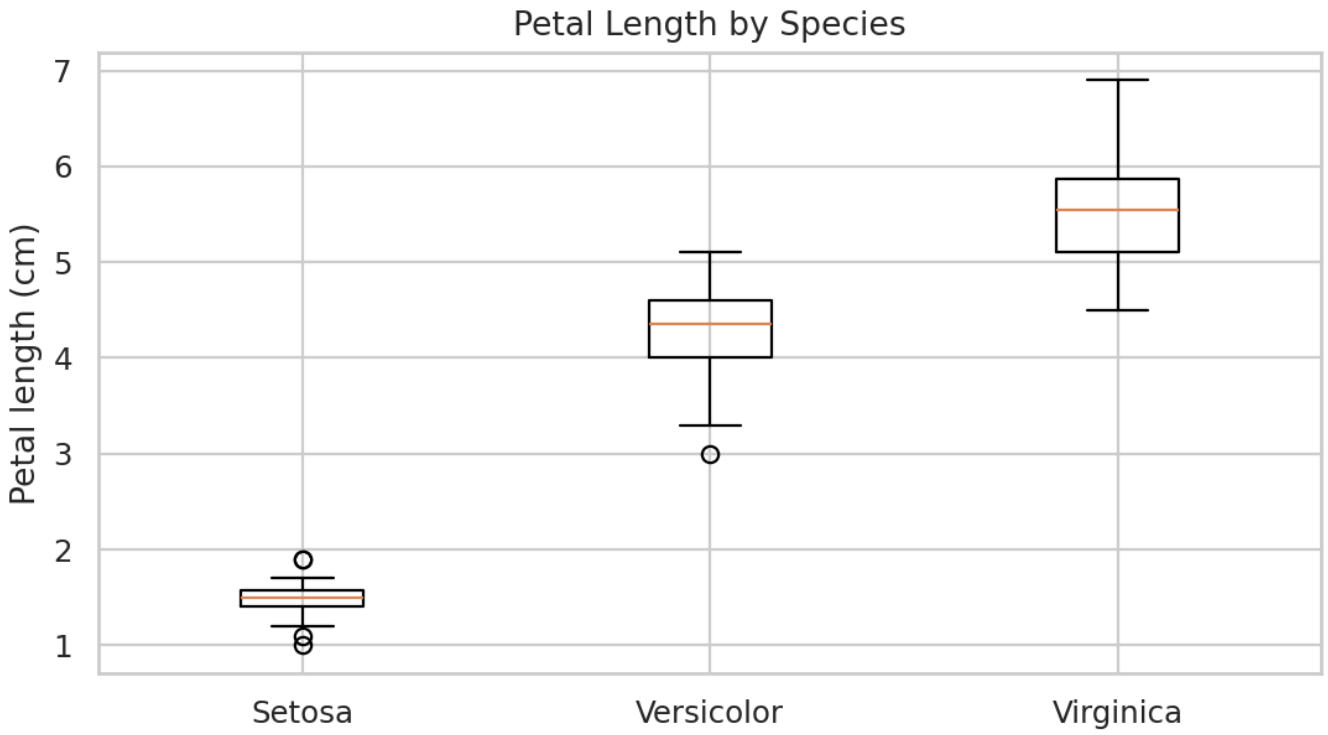
Visualization 2 – Sepal Relationship



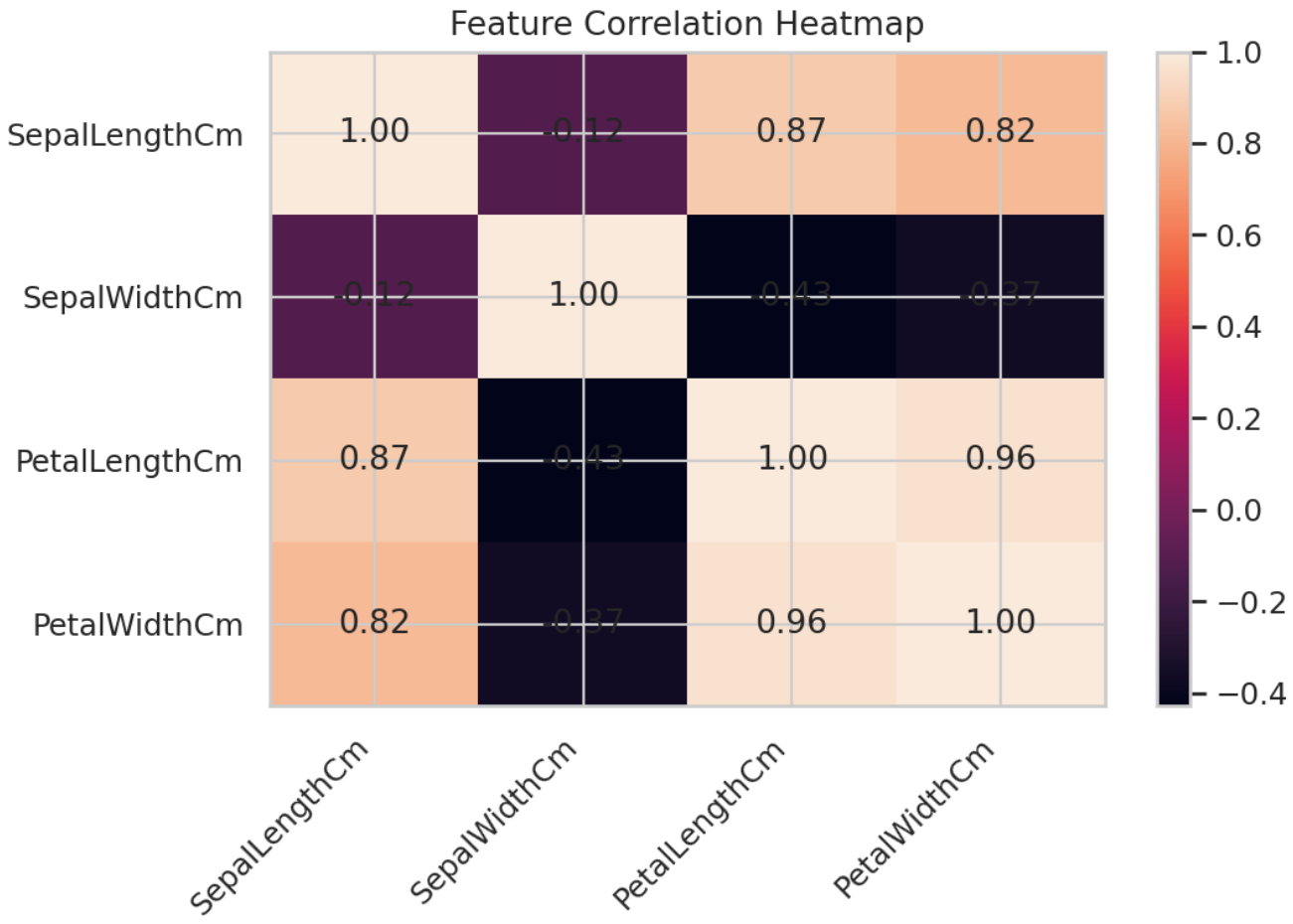
Visualization 3 – Petal Relationship



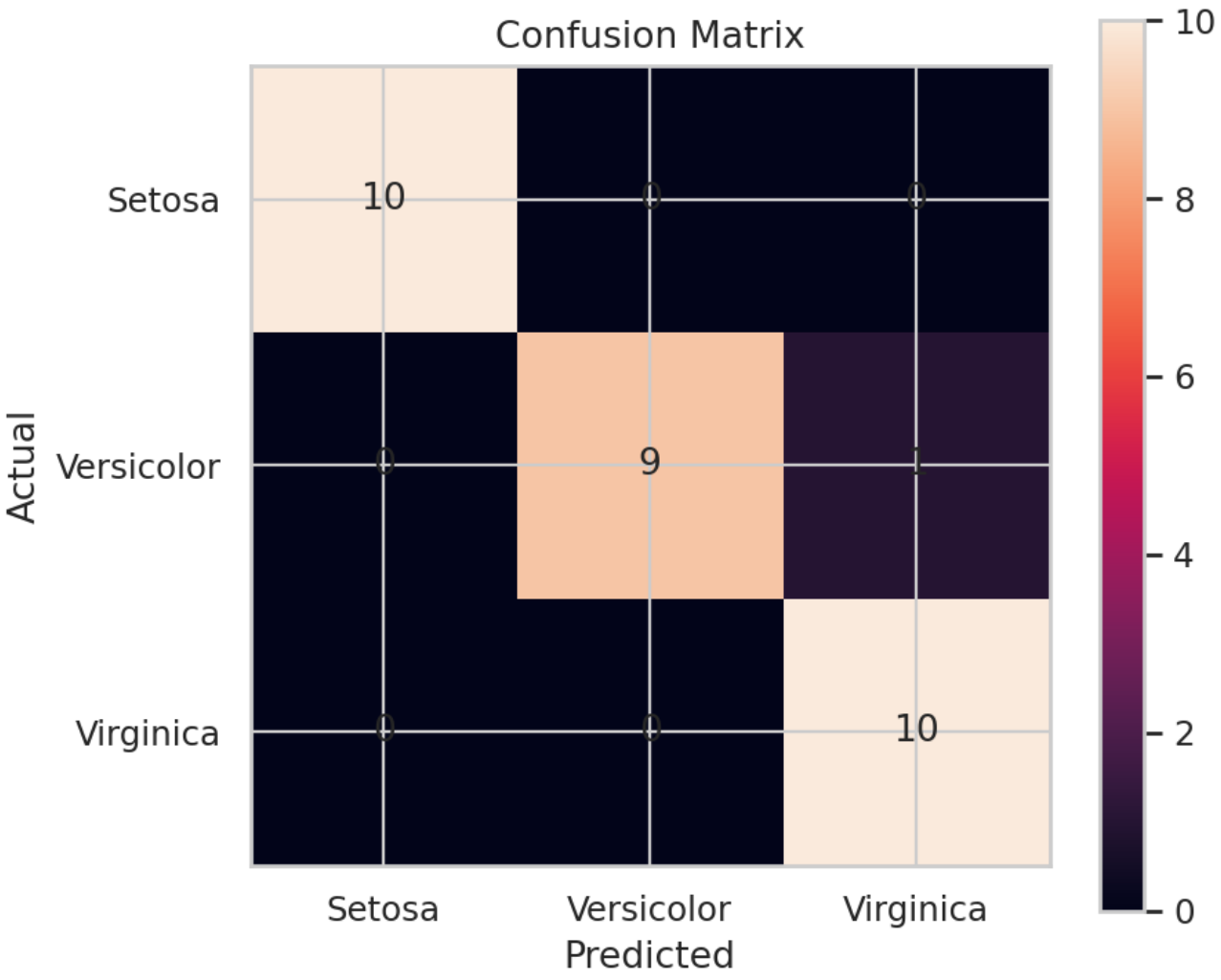
Visualization 4 – Petal Length Box Plot



Visualization 5 – Feature Correlation



Visualization 6 – Confusion Matrix



Model Result

Logistic Regression accuracy: 96.67%.

Complete Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("Iris.csv")
df = df.drop(columns=["Id"], errors="ignore")

# Visualizations
sns.countplot(data=df, x="Species")
plt.title("Iris Species Count")
plt.show()

sns.scatterplot(data=df, x="SepalLengthCm", y="SepalWidthCm", hue="Species")
plt.title("Sepal Length vs Sepal Width")
plt.show()

sns.scatterplot(data=df, x="PetalLengthCm", y="PetalWidthCm", hue="Species")
plt.title("Petal Length vs Petal Width")
plt.show()

sns.boxplot(data=df, x="Species", y="PetalLengthCm")
plt.title("Petal Length by Species")
plt.show()

features=["SepalLengthCm","SepalWidthCm","PetalLengthCm","PetalWidthCm"]
sns.heatmap(df[features].corr(), annot=True, fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()
# Model
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report

X=df[features]; y=df["Species"]
X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.20,random_state=42,stratify=y)
model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))

sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,fmt="d")
plt.title("Confusion Matrix")
plt.show()
```